

WASP SWE Essay: AI Engineering

Robin Hollifeldt

September 1, 2026

1 Introduction

My research topic is Multimodal Deep Learning. It is a growing paradigm of deep learning research that aims at combining various modalities, e.g. text, image, video, audio and various sensor data. Our world is intrinsically multimodal (at least from a human's perspective), rendering multimodal deep learning models the core engines of next-generation autonomous systems for an enormous portion of application cases like auto-driving, robotics or medical imaging.

From a software engineering's perspective, some of the key issues worth addressing are

1. massive resource consumption, and
2. best practices for integration with downstream tasks.

Resource consumption. Deep learning, especially modern large language models (LLMs) are known for its massive needs of electricity, cooling and compute power. Fusing multiple high-dimensional modalities dramatically inflates the resource requirements, making resource a even bigger problem.

Downstream tasks. Unlike algebra or history, deep learning is a highly applicable subject where many research topics can be directly connected with downstream application. However, there still lack a mature and well-established framework that connect the research with the real world. This can also result in resource inefficiency.

Since my research is tightly related to AI, meanwhile I also use AI in my research, the following of the essay will discuss AI engineering from both the perspectives of a creator and a user.

2 Lecture Principles

We discuss the following principles to address the challenges of silent failures and resource waste in multimodal deep learning. (Lecture 2)

2.1 Property-Based Testing for Robust Input Pipelines

Traditional software testing relies on concrete unit tests, where a developer hardcodes a specific input and asserts a specific output. However, multimodal pipelines are non-deterministic, high-dimensional, and undergo complex, sequential transformations. Writing sufficient manual test cases to cover the infinite permutation of multimodal inputs is practically impossible.

Property-Based Testing (PBT) offers a powerful paradigm shift. Instead of asserting specific outputs for specific inputs, the developer needs to define fundamental rules or properties that must always hold true, regardless of the generated input. Such a framework can, for instance, automatically generate a vast array of random test cases to find inputs that violate these invariants, making the debugging workflow more robust.

In my own research, PBT can be systematically integrated into the pre-processing and fusion modules to catch bugs before training. For instance, regardless of the random image cropping size, visual aspect ratio, or text token length, the fused multimodal tensor almost always projects into a fixed-size embedding space, which allows us to define a property asserting that the output shape is invariant to the dynamic input shapes of individual modalities. By running PBT on data loaders and alignment layers prior to launching training, we can catch subtle coordinate mapping bugs or normalisation glitches, therefore saving many GPU-hours.

2.2 Input Slicing and Equivalence Class Partitioning for Multimodal Validation

In typical software engineering, equivalence class partitioning is used to divide a program’s input domain into classes of data from which test cases can be systematically drawn, assuming that the system behaves similarly for any input within a class. In his article on model quality, Christian Kästner [2] adapts this concept to machine learning as *input slicing*, identifying and isolating sub-populations or specific dimensions of interest within the overall dataset.

In deep learning, evaluating model quality solely using overall average accuracy or a global F1-score can mask deep problems like data imbalance or modality imbalance, e.g. an overall accuracy of 90% can easily mask 100% failure rates on specific critical subsets of the data, or an image recognition model can focus on the background water instead of the actual bird to identify waterfowls. Input slicing requires us to systematically partition our multimodal evaluation datasets into meaningful classes based on meta-data, data structure etc, or create adversarial examples to test the model’s robustness. This allows for targeted data collection, customised augmentation, or architectural adjustments, ensuring the model is robust across all operational domains before it is integrated into a production system. This

is, of course, an active research area of its own and cannot be thoroughly in one paragraph.

3 Guest-Lecture Principles

Albeit programming all day long, I have a rather insufficient background in software engineering, which makes me more interested in the basic principles of the subject. In this section, I explore two highly relevant principles: Systems Engineering V-Model for ML Systems and Workflow Transition to AI-Mediated Work.

3.1 Systems Engineering V-Model for ML Systems

When a research-based machine learning algorithm is to be integrated into a real world system, the mismatches between the design styles and principles can make this process very inefficient. The V-Model, a classical systems engineering framework, offers a structured way of dealing with this problem [5]. The main idea is that every step in the development process should have a corresponding verification or validation step. Requirements, system decomposition, and component design on the left side of the V should therefore be directly connected to unit testing, integration testing, and system-level validation on the right side.

This is particularly relevant to multimodal deep learning because multimodal systems usually consist of many components. A simple system may contain a visual encoder, an a text tokenizer, several preprocessing modules, and one or a few fusion layer connecting everything together. This makes it easy for a small error in one component to propagate through the entire pipeline. Applying the V-Model would mean defining important system requirements early and catch bugs early. The system can then be decomposed into components with clear interfaces, while the corresponding tests are designed at the same time.

3.2 Workflow Transition to AI-Mediated Work

The transition from traditional software development towards AI-mediated workflows changes not only how code is written, but also how requirements are specified, how people collaborate, and how software quality is evaluated.

Before the widespread use of AI coding tools, development was mostly based on manually written code, custom scripts, and manually designed test cases. Requirements tend to exist in both written formats and conventions. Software quality was then ensured through unit tests, reviews, QA procedures, and human approval. With AI-mediated development, parts of these activities are increasingly delegated to AI systems. Consequently, human work moves towards specifying constraints, reviewing generated results, and

designing mechanisms that can detect when the AI system has made a mistake. In my experience, co-working with an AI agent changes my role from writing every line myself towards defining what the code must satisfy and checking whether it actually satisfies these requirements. The generation speed also forces users to think through the specific ideas and communicate them down in a decent structure. However, this also means the coding agents can either save resource if used well, or waste resource if prompted poorly.

4 Data Scientists, Software Engineers, and AI Engineers

4.1 Distinctions and Blurring Boundaries

The Machine Learning in Production book distinguishes data scientists and statistical evaluation from software engineers, where the former mainly focus on data and models, and the latter focus on building reliable and maintainable systems.

I think this distinction is sensible, but the boundary is becoming less clear. Data scientists increasingly need software engineering skills such as testing and reproducibility, while software engineers working with ML need to understand data and probabilistic model behaviour.

4.2 The System-Wide View in Multimodal Deep Learning

This system-wide view is particularly relevant to my research. A multimodal model is only one part of the system. Preprocessing, modality alignment, compute resources, and downstream integration can all determine whether it is actually useful. Therefore, a model with a high benchmark score is not necessarily a good system. In my field, software engineering principles are important precisely because they force us to consider these parts together rather than treating the model in isolation.

4.3 Defining AI Engineering and the Impact of Emerging Tech

I see AI Engineering as partly distinct from both software engineering and data science. It combines ideas from both, but focuses on building reliable systems around AI components that are probabilistic and difficult to specify completely. Software engineers can now build substantial AI functionality without training models, while researchers increasingly need to deal with software architecture, testing, and integration. In this sense, AI Engineering is less a replacement for the existing roles than a growing overlap between them.

5 Paper Analysis

5.1 Paper 1: Mutation-based Consistency Testing for LLM Code Understanding [4]

Core Ideas and SE Importance. The paper proposes Mutation-based Consistency Testing (MCT) to evaluate whether LLMs actually understand the semantics of code. Instead of syntax validation, MCT introduces small mutations that change the program’s behaviour and tests whether the LLM can detect the resulting inconsistency with its natural-language description. In reality, AI-generated code (or content in general) can look completely reasonable while still implementing the wrong behaviour (e.g. by creating fallbacks that we did not ask for), making semantic validation an important part of the development process.

Relation to My Research. For my research, AI coding assistants can generate much of this code, but a small semantic error may only become visible after an expensive training run. MCT therefore points to a way of testing the AI-assisted development process itself, rather than simply trusting that generated code is correct.

Integration into a Larger AI-Intensive Project. Consider a (fictional) clinical model that combines chest X-rays, patient records, and recorded consultations to produce a draft diagnostic report. Much of the preprocessing and training code could be generated with AI coding assistants. MCT could be integrated into the pipeline to introduce controlled mutations into this code and test whether the coding model can identify them before the system is sent to an expensive training run.

Adaptation of My Research. The MCT paper motivates me to make the software surrounding my models more testable. In particular, I would like to investigate how mutation-based testing could be combined with the Property-Based Testing approach discussed earlier, especially for modality-specific preprocessing and fusion code.

5.2 Paper 2: Visual Program Repair and Cross-Modal Reasoning [1]

Core Ideas and SE Importance. The paper proposes **GUIRepair**, which uses multimodal models to repair GUI bugs using both visual and textual information. A screenshot can contain information that is difficult to express in a textual bug report, so the system uses images together with source code and bug descriptions to reproduce a problem and then validates the proposed repair through the resulting GUI. I find this interesting because the validation itself becomes multimodal: instead of only asking whether the code is syntactically correct, the system checks whether the actual visible behaviour has been fixed.

Relation to My Research. GUIRepair is a concrete example of a downstream task where different modalities must be combined for reasoning rather than simply aligned. The system needs to connect a visual symptom with textual information and source code, and then use the result of one modality to validate another. For my research, this shows that multimodal learning can be valuable precisely when information from one modality is necessary to interpret or verify another.

Integration into a Larger AI-Intensive Project. The same clinical system could use a GUIRepair-style approach for its clinical interface. For example, a visual annotation covering an important label could be difficult to detect using conventional software tests, but a multimodal model could compare the intended and rendered interfaces and identify the problem.

Adaptation of My Research. This paper suggests that more emphasis should be placed on downstream tasks that require actual cross-modal reasoning. Rather than evaluating my models only through global alignment or benchmark accuracy, I would like to investigate whether information from one modality can be used to reason about and validate another. This would also encourage more modular architectures, where individual modalities can be tested separately while still supporting deeper interactions during fusion.

6 Evidence, Originality, and GenAI Transparency

6.1 Source Trustworthiness and Validation

The trustworthiness of the main references are evaluated as follows:

- Machine Learning in Production is a textbook [3], thus very likely to be reliable.
- The CAIN 2024 papers by Wu [5] and Ma et al. [4] are peer-reviewed publications from a premier joint ACM/IEEE conference on AI engineering, thus also reliable.
- The work by Huang et al. [1] is a state-of-the-art preprint. While it represents cutting-edge technical innovation in visual program repair, it leaves some pace for critical thinking.
- Christian Kästner’s blog post [2] can be seen as a high-quality expert commentary; although not peer-reviewed or published by a publisher, it can still be considered as community discussion.

6.2 Generative AI usage declaration

I used AI tools for brainstorming, extensive discussion and clarification of concepts based on the course material and my own research field. AI was also

used for navigating publications, citation formating and evaluation relevance to the topic.

References

- [1] Kai Huang, Jian Zhang, Xiaofei Xie, and Chunyang Chen. Seeing is fixing: Cross-modal reasoning with multimodal llms for visual software issue fixing, 2025.
- [2] Christian Kästner. Model quality: Slicing, capabilities, invariants, and other testing strategies, 2021. Blog post.
- [3] Christian Kästner. *Machine Learning in Production: From Models to Products*. Carnegie Mellon University, 2025. Book chapter (Chapters 1 and 2).
- [4] Ziyu Li and Donghwan Shin. Mutation-based consistency testing for evaluating the code understanding capability of llms. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering - Software Engineering for AI (CAIN 2024)*, 2024.
- [5] Jie JW Wu. An exploratory study of v-model in building ml-enabled software: A systems engineering perspective. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering - Software Engineering for AI (CAIN 2024)*, 2024.